

Article

Computational Thinking Measurement of CS University Students

Raquel Hijón-Neira ^{1,*} , Celeste Pizarro ² , John French ³ , Daniel Palacios-Alonso ¹  and Emre Çoban ⁴ 

- ¹ Computer Science Department, Rey Juan Carlos University, 28933 Móstoles, Spain; daniel.palacios@urjc.es
² Applied Mathematics Department, Rey Juan Carlos University, 28933 Móstoles, Spain; celeste.pizarro@urjc.es
³ Computer Science & Applied Physics Department, Atlantic Technological University, H91 T8NW Galway, Ireland; john.french@atu.ie
⁴ Computer Technology Department, Ardahan University, 75000 Ardahan, Türkiye; emrecoban@ardahan.edu.tr
* Correspondence: raquel.hijon@urjc.es

Featured Application: Altogether, UniCTCheck, a method for measuring the main components of computational thinking of CS university students, was employed by using the following two instruments: firstly, a web application, CTScore, to precisely measure seven of the main components of computational thinking (pattern recognition, creative thinking, algorithmic thinking, problem solving, critical thinking, decomposition, and abstraction); secondly, a psychometric scale, CTProg, to measure CT programming concepts skills (basic directions and sequences, conditionals, loops, functions, and data structures).

Abstract: The measurement of computational thinking ability among computer science (CS) university students is of paramount importance. This study introduces UniCTCheck, a novel method designed to assess the main components of computational thinking in CS students. Utilising two key instruments, namely, the web application CTScore and the psychometric scale CTProg, this research aims to precisely evaluate seven core components of computational thinking and six programming concepts skills essential for CS students. The study, conducted at Rey Juan Carlos University and Atlantic Technological University Galway, involved a diverse sample of students from different year levels and programme specialisations. Through a rigorous research design, including sampling strategies and data collection tools, this study seeks to address critical research questions related to the measurement of variations in students' computational thinking and programming skills by gender, university level, and location. By shedding light on the significance of computational thinking and programming in the educational realm, this research contributes to the existing literature and underscores the essential role of computational skills in the modern era.

Keywords: computational thinking; programming; performance-based assessment platform; interactive web app; evaluation methods; psychometric scale; student evaluation



Citation: Hijón-Neira, R.; Pizarro, C.; French, J.; Palacios-Alonso, D.; Çoban, E. Computational Thinking Measurement of CS University Students. *Appl. Sci.* **2024**, *14*, 5261. <https://doi.org/10.3390/app14125261>

Academic Editor: Yu Liang

Received: 15 May 2024

Revised: 5 June 2024

Accepted: 11 June 2024

Published: 18 June 2024



Copyright: © 2024 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

The act of methodically analysing a particular issue and coming up with an answer is known as computational thinking. It consists of a particular type of skill group that is predicated on approaches and abilities for solving problems. The goal of this skill set is to comprehend, analyse, and create solutions for difficult situations. According to [1–3], people with computational thinking abilities are supposed to be able to process information like computer scientists and use this method of thinking in all fields. Consequently, it may be concluded that the development of 21st century skills and computational thinking ability are closely related. According to [4], computational thinking aids in the understanding of computable problems as well as the selection of appropriate tools and approaches for problem solving.

Almost every field of study appears to have some connection to computational thinking, from biology to machine learning [5,6]. The term “computational thinking” was coined to refer to problem-solving techniques that also make it possible to use modern technologies efficiently in a variety of contexts [1]. Organisations have acknowledged the necessity for problem solving and the efficient use of information and communication technology, notwithstanding the lack of agreement among them. In this context, information societies—the age and society of today—need computational thinking. In order for people to coexist peacefully in society, it is vital to identify the knowledge, skills, and abilities that we require. Such studies are necessary for addressing the obstacles that societies—students in particular—will encounter in the years ahead. It is observed that common components are employed in the literature, despite the fact that the term computational thinking and its components remain unaltered [7–9].

In addition to explorations of computational thinking and its elements, another subject of research involves how to quantify these elements. It has been observed in previous research that several measurement instruments are used to assess computational thinking. Numerous other techniques have been used with various approaches, including scales, portfolio studies, programming, multiple choice exams, task-based tests, observations, and rubrics [10–12]. However, some believe that the current approaches of measuring computational thinking may not be adequate. In fact, a student who performs well on examinations could nevertheless be an incompetent or insensitive algorithm creator, according to [13]. The author of [13] further underlined that, while we do not assess pupils’ competencies or sensitivities, we do know what they know. He advised treating the components of computational thinking as a talent and grading them accordingly. He stated that the assessment of abilities in school is a pretty typical practice and cited examples that included language, athletics, music, and theatre. By using this study to create a valid and trustworthy performance-based assessment tool, the framework seeks to quantify computational thinking abilities. Upon reviewing the literature, it becomes evident that the studies used to assess computational thinking have several drawbacks, including being confined to a narrow framework and relying on computer software in a traditional and dull manner [14,15]. Moreover, the measurement of computational thinking abilities in many of these investigations requires the expertise of an expert.

Psychometric scales are among the most widely used instruments for assessing computational thinking [12,14–17]. However, the foundation of psychometric measures is the idea that a subject provides accurate and complete information [18]. It is anticipated that the person has an inner vision, nonetheless [19]. Performance-based assessments can be more appropriate because psychometric scales have these drawbacks and computational thinking components should be treated as skills [13]. Furthermore, research on the measurement of computational thinking using performance-based assessments has highlighted the significance of receiving feedback at the appropriate moment [20,21]. Performance-based assessments offer numerous benefits, including their capacity to evaluate intricate and advanced cognitive abilities, and their connection to the educational process [22–24].

As computational thinking is a relatively new and growing skill, there is currently no agreement on its description or its constituent parts [6,8,9,25,26]. Therefore, the current theoretical framework was used to conduct this investigation. The purpose of this study was to find a method to effectively measure computational thinking abilities in CS university students. However, numerous studies in the literature have found that the only instruments available for measuring computational thinking are limited perception–attitude scales, multiple choice tests, or simple coding tests [10–12,14–16,27]. Therefore, a performance-based platform, CTScore, for evaluating computational thinking abilities was created in [28]. Additionally, it has been determined that performance-based assessment instruments provide more accurate results than psychometric scales. It is stressed that computational thinking should not be viewed as theoretical knowledge, a product or portfolio, or an attitude or perception, but rather as a talent or set of skills.

The hypotheses of this research are as follows:

H1. *It is possible to develop UniCTCheck, a method for measuring the main components of computational thinking, in the context of CS university courses.*

H2. *It is possible to precisely measure seven computational thinking components (pattern recognition, creative thinking, algorithmic thinking, problem solving, critical thinking, decomposition, and abstraction) by using CTScore.*

H3. *It is possible to use a psychometric scale, CTProg, for measuring CT programming concepts skills that all CS university students should address sooner rather than later (basic directions and sequences, conditionals, loops, functions, and data structures).*

Given the paucity of research and advancement on the topic, it is believed that computational thinking has a vital role nowadays and that this study will significantly add to the body of literature. Therefore, following on from on the hypotheses above, we focus on addressing the following research problems in this work:

- RQ1: Does UniCTCheck measure the main components of computational thinking and programming for CS university students?
- RQ2: Is there a relationship between students' computational thinking skills and programming abilities?
- RQ3: Do students' computational thinking skills and programming abilities vary by university level?
- RQ4: Do students' computational thinking skills and programming abilities vary by university location?
- RQ5: Do students' computational thinking skills and programming abilities vary by gender?

1.1. Computational Thinking

Despite lacking a precise definition, computational thinking has a rich history dating back to the early days of computer science. The phrase “computational thinking” first appeared in George Polya’s 1940 book *How to Solve It*, which discussed mental disciplines and techniques (Denning, 2017 [13]; Tedre and Denning, 2016 [29]). Then, in 1960, Alan Perlis coined the term “algorithmizing”. Perlis predicted that computers would automate procedures in every industry by using this word to describe practice and thought (Katz, 1960 [30]). Three of the field’s pioneers—Newell, Perlis, and Simon—discussed computer science in a 1967 publication. This published work considers “algorithmic thinking” to be a phrase that sets computer science apart from other disciplines [31]. In 1980, Seymour Papert introduced the phrase “computational thinking” in his book *Mindstorms: Children, Computers, and Powerful Ideas* [32]. Even though Papert created the phrase first, no one had previously defined computational thinking in the very beginning. Papert used the phrase computational thinking a lot in his publications in the years that followed [33–35]. In the 1950s and 1960s, in particular, “algorithmic thinking” was the term used before “computational thinking” [36–38]. One of the fathers of computer science, Edsger Dijkstra, defined three characteristics of algorithmic thinking: the ability to move back and forth between semantic levels; the ability to reveal one’s own form and concept while solving problems; the ability to use one’s mother tongue as a bridge between informal problems and formal solutions [39].

The term “computational thinking” was originally used in Jeannette M. Wing’s widely read paper, Computational Thinking [27,40–42]. According to Wing, the phrase computational thinking encompasses problem solving, system design, and behavioural analysis. Additionally, she said that not only computer scientists but everyone should use computational thinking [1]. Despite being a seminal contribution to the area, Wing’s 2006 definition has drawn a great deal of criticism [29]. Wing’s definition has drawn criticism for using an imprecise and confusing phrase [26]. Moreover, her unverified assertions regarding the all-encompassing advantages of computational thinking have been deemed audacious [40].

Wing's article Computational Thinking and Thinking about Computing from 2008 brought the phrase computational thinking back into focus. Wing compared analytical and computational thinking in this paper. She pointed out that scientific, engineering, and mathematical reasoning are all entwined with computational thinking. Furthermore, she underlined once more that computational thinking applies to everyone and in all contexts [2]. A variety of publications have been released in the years that followed to provide a clearer definition of the phrase computational thinking, and Thinking About Computational Thinking is one of them [43]. Rather than expressing criticism, Lu and Fletcher have declared their support for Wing's 3R (writing, reading, and math) rule and for placing computational thinking at the centre of computer science. Nonetheless, debates over Wing's concept have persisted in the literature. Following this, Canadian computer scientist Alfred V. Aho wrote a piece on computational thinking for Alan Turing's 100th birthday celebration in 2012. In this piece, Aho underlined the importance of the precise and intelligible identification and the significance of settling on a scientific field's nomenclature for the sake of comprehension [44].

On one hand, educators have contributed to Wing's development of the term computational thinking as well as Papert's traditional formulation through hundreds of workshops, committees, studies, polls, and public evaluations [13,45]. On the other hand, it is impossible to discuss a widely agreed-upon definition and list of elements for computational thinking. Communities like the CSTA, CAS, and ISTE, which are regarded as leading and respected organisations, have not been able to come to an agreement on the components of computational thinking [46–48]. This disagreement is in addition to the divergent opinions of researchers regarding the definition of computational thinking. However, based on a review of the literature, it appears that there are recurring themes that are well acknowledged under the heading of the suggested definition [9]. According to educators, computational thinking is particularly beneficial for enhancing logical reasoning, growing analytical thinking, and problem-solving abilities [49].

Despite its rich history and significance in computer science, computational thinking (CT) lacks a universally accepted definition. Initially discussed by George Polya, and later popularised by Seymour Papert and Jeannette M. Wing, CT encompasses problem solving, system design, and behavioural analysis. Wing's broad claims about CT's applicability have sparked debate, with critics highlighting the need for precise terminology. Leading organisations and educators continue to refine the concept, emphasising themes like logical reasoning, analytical thinking, and problem solving, though consensus on a definitive list of CT components remains elusive.

1.2. Components of Computational Thinking

Problem-solving techniques are unquestionably a part of computational thinking [3,8]. Computational thinking is described by Wing in her well-known article as problem solving, system design, and behavioural analysis. She also said that, as opposed to thinking like a computer, computational thinking involves addressing issues in an efficient and original way. Nonetheless, she stressed that computational thinking encompasses methods like recursion, systematic testing, problem analysis, abstraction, and problem re-explanation [1]. The concept of computational thinking and its constituent parts have sparked a great deal of discussion in the years that followed. The components have been recommended by communities, educators, and pioneering computer scientists (see Table 1). Computational thinking includes testing hypotheses, managing data, parallelism, abstraction, and debugging according to the National Research Council [50]. Barr and Stephenson provided an organised approach that allowed them to separate computational thinking into two types of concepts and abilities. According to [51], the capabilities include automation, abstraction, simulation, issue decomposition, data collecting, analysis, and representation. In [52], the authors categorised computational thinking into the following three categories: views, applied methods, and concepts. However, this categorisation only pertains to programming. Gouws et al. separated computational thinking into two categories in order to develop a framework. A set of abilities related to computational thinking is included in the first

of these aspects. Processes and transformations, models and transformations, logic and inference, patterns and algorithms, assessments, and enhancements are some examples of these. The idea that various skills—such as identifying, comprehending, applying, and assimilating—have varying degrees of experience defines the second dimension [53]. Another study [54] classified computational thinking as abstraction, generalisation, and trial-and-error processes. Similar component recommendations were made in a study that sought to draw a link between learning theories and computational thinking. These elements encompass heuristic scanning, trial-and-error, abstract and divergent thinking, recursion, iteration, collaborative approaches, metacognition, patterns, and synectics [55]. According to a recently released study, computer programmers frequently employ computational thinking as a problem-solving strategy. Problem analysis, pattern identification, abstraction, algorithm creation for answers, and the evaluation of solutions (including debugging) are some of its constituents [56].

Prominent organisations have also released research on computational thinking components, in addition to the academics (see Table 2). Beyond contributing to the communities, studies on computational thinking published by well-known tech giant Google have also added to the body of literature. Pattern recognition, decomposition, pattern generalisation and algorithm design, and abstraction are the four areas into which Google separated computational thinking [57].

Computational thinking (CT) encompasses a range of problem-solving techniques, including recursion, systematic testing, problem analysis, abstraction, and debugging. Key components identified by various scholars and organisations include algorithm building, data management, hypothesis testing, and parallelism. Major frameworks from entities like the National Research Council, ISTE, and Google emphasise similar elements such as pattern recognition, decomposition, abstraction, and algorithm design. Despite differing categorisations, the core skills of CT focus on logical reasoning, problem decomposition, and the efficient creation and evaluation of algorithms.

To measure these components effectively, this study introduces UniCTCheck (Figure 1), a comprehensive framework that includes the CTScore web application for assessing seven distinct CT elements (Figure 1, left) and the CTProg psychometric scale for evaluating five computational programming concepts (Figure 1, right). This innovative approach aims to provide a precise and holistic evaluation of computational thinking skills in university-level computer science students.

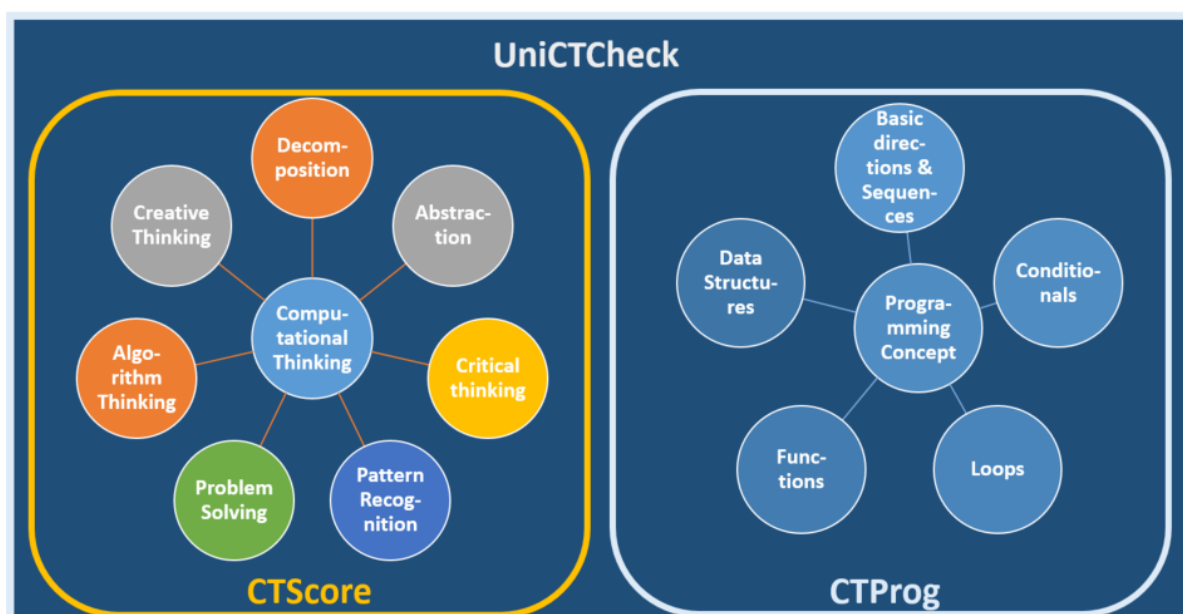


Figure 1. The components of UniCTCheck: left, CTScore; right, CTProg.

Table 1. Computational thinking skills defined by the researchers.

Wing, J. [1]	Dierbach, C. et al. [58]	Berland, M. and Lee, V.R. [59]
Algorithm building	Algorithm building	Algorithm building
Conditional logic	Problem decomposing	Conditional logic
Debugging	Developing a computational model	Debugging
Distributed processing	Evaluating a problem	Distributed computation
Simulation		Simulation
Lee, I. et al. [41]	Brennan, K and Resnick, M. [52]	Selby, C. and Woollard, J. [9]
Abstraction	Abstracting and modularising	Abstraction
Analysing the model	Creative thinking	Algorithm building
Automation	Debugging	Evaluating solutions
Defining a problem	Decomposing	Problem decomposing
Understanding/solving problems	Reusing and remixing	Generalisations
	Testing and debugging	Problem solving

Source [60].

Figure 1 illustrates the two main components of the UniCTCheck framework, including CTScore and CTProg. The left side of the figure represents CTScore, a web application designed to measure seven core components of computational thinking (CT). These components are pattern recognition, creative thinking, algorithmic thinking, problem solving, critical thinking, decomposition, and abstraction. The right side of the figure shows CTProg, a psychometric scale aimed at assessing critical programming concepts essential for computer science students. These programming concepts include basic directions and sequences, conditionals, loops, functions, and data structures. Together, CTScore and CTProg provide a comprehensive evaluation method for assessing the computational thinking and programming skills of university-level computer science students. Both were offered in English (<https://emrecoban.com.tr/bd/> and <https://forms.office.com/e/JXDaTriwi9>) and Spanish (<https://emrecoban.com.tr/bd/es/> and <https://forms.office.com/e/3mbvMXw1vX>) (accessed on 10 June 2024).

Table 2. Computational thinking components defined by the communities.

Community	Framework	Components
ISTE	Standards for Students in CT (2016) [48]	Leverage the power of technological methods to develop and test solutions Collect the data Analyse the data Represent the data Decomposition Abstraction Algorithms Testing Parallelisation Simulation
CAS	Concepts of CT (2015) [61]	Logical reasoning Algorithmic thinking Decomposition Generalisation Patterns Abstraction Representation Evaluation
CSTA	Concepts of CT (2011) [47]	Formulating problems for a computational solution Logically organising and analysing the data Abstractions, including models and simulations Algorithmic thinking Evaluation for efficiency and correctness Generalising and transferring to other domains

Denning [13].

2. Materials and Methods

2.1. Research Design

A quantitative study was conducted to measure the computational thinking ability of students studying the subjects “Visual Programming” and “Games for web and social networks” of the Video Games Degree of the ETSII, URJC, and of students studying the subjects of “Software Design and Program Development” and “Software Engineering” of the B.Sc. in Software Development at Atlantic Technological University (ATU), Galway, Ireland. A first-year subject and a final-year subject were selected at each university. The students used an interactive web application called CTScore for measuring computational thinking abilities [28], and also a psychometric programming scale (CTProg) to measure programming skills as a complement to the CTScore web application. Figure 2 shows the outline of the experimental design.

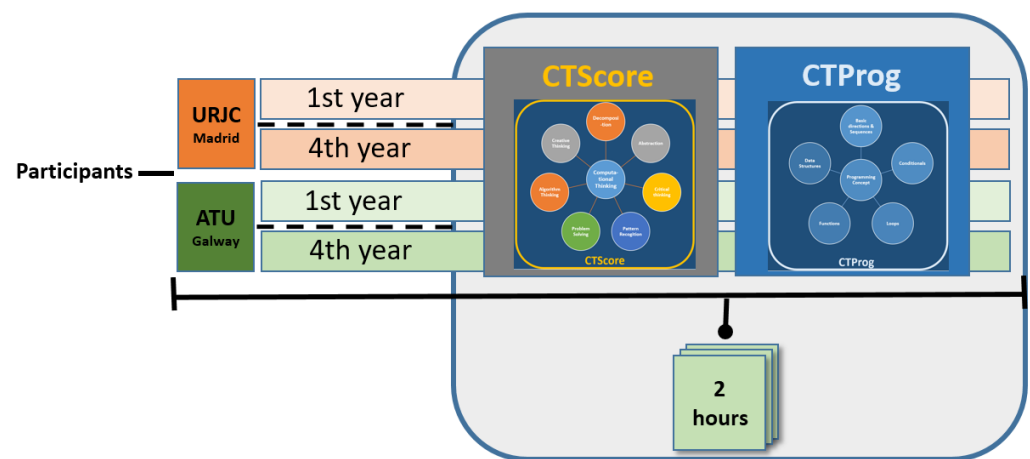


Figure 2. Outline of the experiment settings and components.

2.2. Sampling

The sample for this study was drawn from students studying in two European universities, Universidad Rey Juan Carlos (URJC) in Madrid, Spain, and Atlantic Technological University (ATU) in Galway, Ireland. In both settings, the students were studying a Computer Science degree programme in which the researchers teach. At URJC, of the 75 students enrolled in the 1st year subject “Introduction to Programming”, 63 completed the 4 tests (19 females, 44 males, 0 other). Of the 90 URJC students taking the 4th year subject “Games for the web and social networks”, 80 students completed the 4 tests (14 female, 65 males, 1 other). Both subjects are part of the Design and Development of Video Games degree programme at URJC. At ATU Galway, from a group of 25 students enrolled in the 1st year subject “Software Design and Program Development”, 15 students completed the 4 tests (1 female, 13 males, 1 other). Of the 30 students enrolled in the 4th year subject “Software Engineering”, 19 students completed the 4 tests (1 female, 18 males, 0 other). Both subjects are part of the B.Sc. in the Software Development degree programme at ATU Galway.

2.3. Ethics Committee

The Research Ethics Committee of Rey Juan Carlos University has favourably evaluated this research project, with internal registration number 1909202332223 to pursue the research at both universities (URJC and ATU) under the same research project.

2.4. Data Collection Tools

UniCTCheck collects data from two tools, CTScore and CTProg. In the following subsections, their scope and characteristics will be explained.

2.4.1. CTScore

CTScore is a validated interactive web application for measuring computational thinking abilities in students [31]. In this paper, the web app is offered in English <https://emrecoban.com.tr/bd/> (accessed on 10 June 2024) and Spanish <https://emrecoban.com.tr/bd/es/> (accessed on 10 June 2024) for the first time. CTScore measures seven computational thinking components, namely, abstraction, pattern recognition, critical thinking, decomposition, creative thinking, algorithmic thinking, and problem solving, by asking students to answer 12 questions (see Figure 3). During the evaluation, students were evaluated automatically with the same scoring key, where a maximum of 10 points and a minimum of 0 (zero) points could be obtained from each question. Accordingly, each question determines the total score of the component in which it is located. For example, since the abstraction component contains two questions, the maximum score that students can achieve in this component will be 20 points.

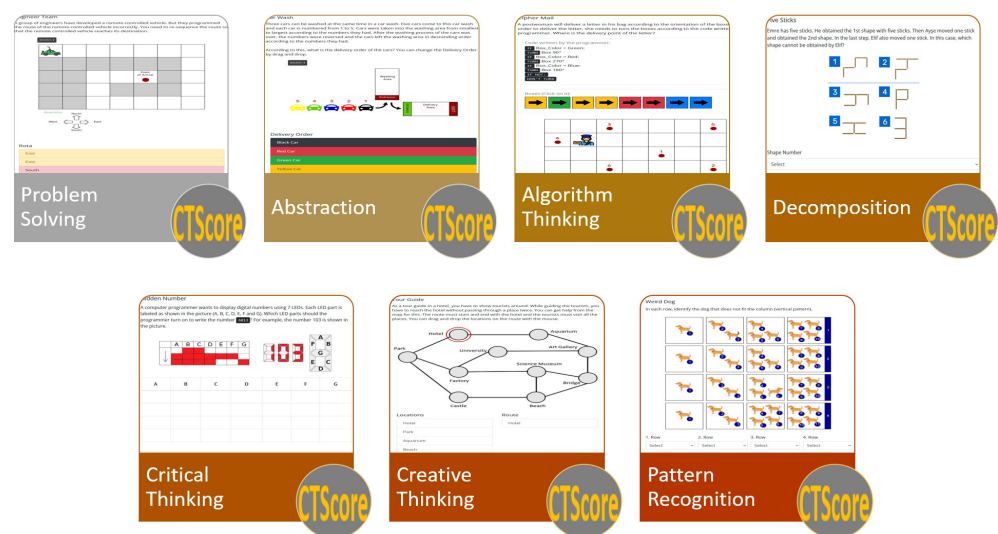


Figure 3. Example questions for each component of CTScore.

2.4.2. CTProg

CTProg is a psychometric scale test created by the authors to measure the conceptual understanding of 5 core programming fundamentals, not depending on any particular programming language. It is formed by 14 questions, providing a schematic block programming language that university students can understand independently of their knowledge of specific programming languages. The tests were used with university students and revised by the researchers over the course of three academic years (2017–2018, 2018–2019, and 2019–2020), continually evolving during this period. The main concepts addressed by the 14 questions included basic directions and sequences, conditions, loops, functions, and data structures (arrays and matrices). CTProg is offered both in English (<https://forms.office.com/e/JXDvTriwi9>) (accessed on 10 June 2024) and Spanish (<https://forms.office.com/e/3mbvMXw1vX>) (accessed on 10 June 2024). During the evaluation, students were evaluated automatically with the same scoring key, where a maximum of 1 point and a minimum of 0 (zero) points could be obtained from each question. Accordingly, each question determines the total score of the programming concept in which it is located (see Figure 4). Each of the main concepts addressed contain two questions, but the loop concept contains four questions; therefore, the maximum score that students can achieve in this component is 14 points.

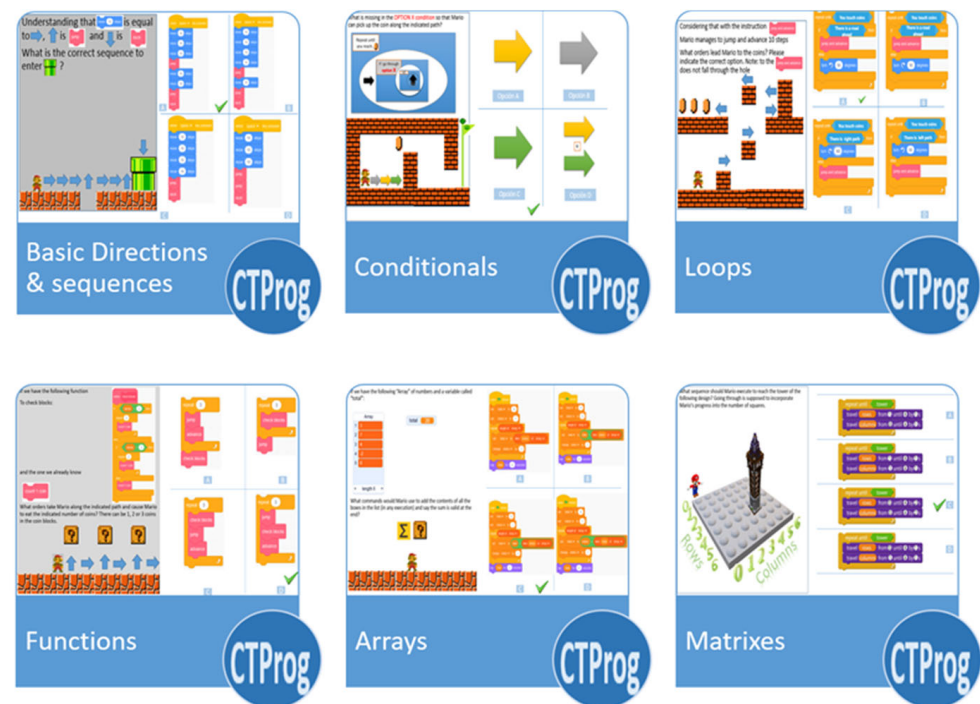


Figure 4. Example questions for each component of CTProg.

3. Results

This section will show the most relevant results of the study. This focuses on general results, and later we will carry out a detailed study of the different dimensions.

3.1. Relations among Variables

Firstly, the possible relationship between the different variables studied was analysed. As the variables do not follow a normal distribution (Shapiro–Wilk test with a p -value < 0.01), a possible linear correlation was analysed using Kendall's Tau-b. Table 3 marks the significant linear correlations in black. Thus, the following can be seen:

- There is a positive linear correlation between the duration of the test and the total score on the programming test (CTProg). This indicates that the more time spent by the students in answering the test resulted in a higher performance.
- There is a positive linear correlation between the total score on the programming test (CTProg) and the total score on the CT test (CTScore). This demonstrated that students that do well on the programming test also do well on the CT test. Thus, it can be said that programming knowledge leads to CT skills acquisition.
- There is a positive linear relationship between students' knowledge of for loops (PLOFOR) with their knowledge of conditionals (PCON) and repeat loops (PLOREPEAT). Thus, it can be said that students learning any type of loop leads to their learning of conditionals, and vice versa.
- There is a positive linear relationship between the knowledge of the array data structure (PARR) with students' knowledge of conditionals (PCON) and all types of loops, repeat loops (PLOREPEAT), and for loops (PLOFOR). Therefore, it can be deduced that students that are familiar with more advanced programming concepts such as arrays have to have previously mastered conditionals and loops.
- There is a positive linear relationship between students' knowledge of the matrix data structure (PMAT) with their knowledge of for loops (PLOFOR) and arrays (PARR). Therefore, the comprehension of two-dimensional arrays (matrices) requires the knowledge and understanding of one-dimensional arrays and for loop structures to work effectively with them.

- There is a positive linear relationship between the CT skill of problem solving (Problemsolving) with the CT skill of algorithmic thinking (AlgorithmicThinking). This shows that students with a good problem-solving ability do well on the algorithmic thinking test, and vice versa.
- There is a positive linear relationship between the CT skill of abstraction (Abstraction) with the CT skills of algorithmic thinking (AlgorithmicThinking), decomposition (Decomposition), and creative thinking (CreativeThinking). As a consequence, students with a higher capacity for abstraction perform as well or better with other CT skills such as algorithmic thinking, decomposition, and creative thinking.
- There is a positive linear relationship between the CT skill of algorithmic thinking (AlgorithmicThinking) with the CT skills of decomposition (Decomposition) and pattern recognition (PatternRecognition). As shown, students overperforming in algorithmic thinking also do well in CT skills like decomposition and pattern recognition.

Table 3. Tau-b coefficients for each pair of variables. Those which are significant are marked in bold (*: $p < 0.05$; **: $p < 0.01$).

		Duration		CTProg		CTScore	
Duration		1		0.179 *			
CTProg		0.179 *		1		0.398 **	
CTScore		0.082		0.398 **		1	
	PBDS	PCON	PLOREPEAT	PFUNC	PLOFOR	PARR	PMAT
PBDS	--						
PCON	−0.014	--					
PLOREPEAT	−0.057	0.188	-- **				
PFUNC	0.129	0.126	−0.123	--			
PLOFOR	0.156	0.236 *	0.088 **	0.115	--		
PARR	0.193	0.219 **	0.094 **	0.200	0.366 **	-- **	
PMAT	0.102	0.072	−0.013	0.155	0.139 *	0.113 *	
	Problem Solving	Abstraction	Algorithmic Thinking	Decomposition	Critical Thinking	Pattern Recognition	
Problemsolving	--						
Abstraction	0.099	--					
AlgorithmicThinking	0.156 *	0.266 **	--				
Decomposition	0.086	0.181 **	0.239 **				
CriticalThinking	0.037	0.067	0.100	0.046			
CreativeThinking	0.110	0.152 *	0.059	0.098	0.116		
PatternRecognition	0.112	0.076	0.132 *	0.093	0.075		

3.2. Comparison between Courses

The scores for each variable are compared between the first and fourth years. Firstly, they are compared by the general score on either test, the CT test (CTScore) and the programming test (CTProg); then, they are compared by university, Rey Juan Carlos University and Atlantic Technological University.

3.2.1. General Results

Figure 5 shows the box plots for the CTScore and CTProg variables differentiated between the first and fourth years. As can be seen in Table 4, the central values of both variables are maintained in both years, with a greater dispersion being perceived for both in the fourth year. Using the Mann–Whitney U test (Table 5), it can be seen that these minimal differences are, in effect, not significant, so it can be ensured that there are no significant differences between both courses.

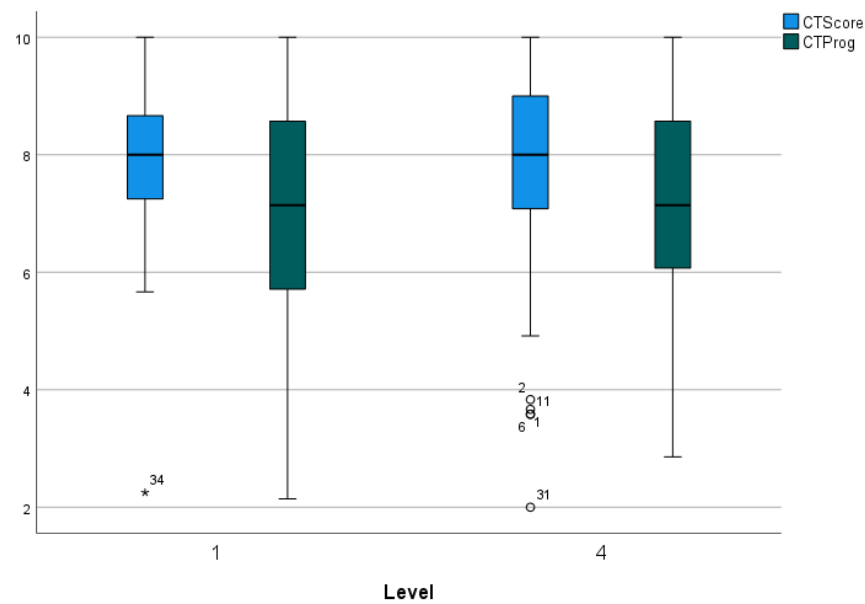


Figure 5. Box plots for the CTScore and CTProg total scores by years (1st and 4th) in general.

Table 4. Descriptive analysis for the CTProg and CTScore.

	CTProg		CTScore	
	1st	4th	1st	4th
n	78	99	78	99
Mean	7.03	7.09	7.81	7.85
Median	7.14	7.14	8.00	8.00
SD	1.54	1.74	1.27	1.54

Table 5. A non-parametric test (Mann–Whitney U test) between the 1st and 4th level.

	CTProg	CTScore
U	3960	4091
p-value	0.768	0.680

Now, the focus is on observing the differences between each of the variables studied. The following graphs (Figure 6, top and bottom) show the median of each of them as a representative of the central tendency in the data. The median is used instead of the mean, since there is no symmetry in the distribution of the data, with the median being a more robust measure. Although they are not statistically significant, changes can be seen between the first and fourth years in the variable for the programming concepts of repeat loops (PLOREPEAT), functions (PFUNC), arrays (PARR), and matrices (PMAT).

3.2.2. Rey Juan Carlos University (URJC)

This part of the study focuses on Rey Juan Carlos University (URJC). In this case (see Figure 7), the values obtained are similar in the first and fourth years for the two variables, the CT test (CTScore) and the programming test (CTProg). In the case of CTScore, the median has increased its value slightly, being 8 in the first year and 8.13 in the fourth year. Dispersion has clearly increased. In the case of CTProg, the median has increased its value, going from 7.14 to 7.85 from the first to the fourth year, respectively. Again, the dispersion in the data has increased.

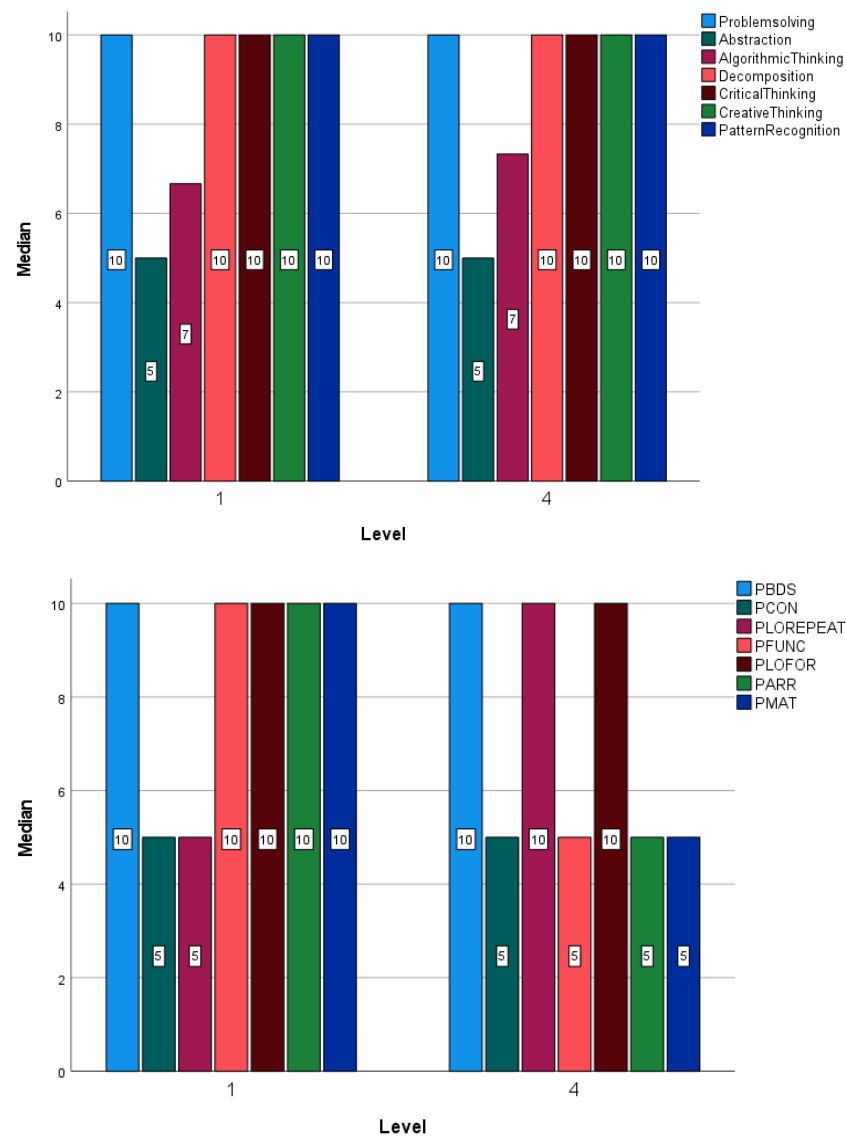


Figure 6. Bar charts of the CT skills in CTScore (top) and the programming test in CTProg (bottom) by years (1st and 4th) in general.

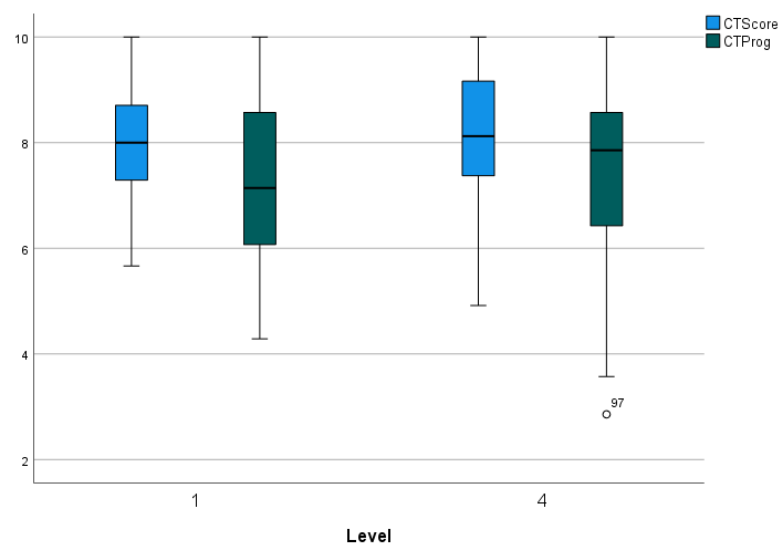


Figure 7. Box plots for the CTScore and CTProg total scores by years (1st and 4th) at URJC.

In the case of the bar graphs (see Figure 8), very little change is observed in the medians, depending on the course, for only the algorithmic thinking variable (AlgorithmicThinking) from CTScore, where it goes from 7 to 10. In the case of the programming variables, the median has increased in the conditionals (PCON) and repeat loops (PLOREPEAT) and decreased in the matrices (PMAT).

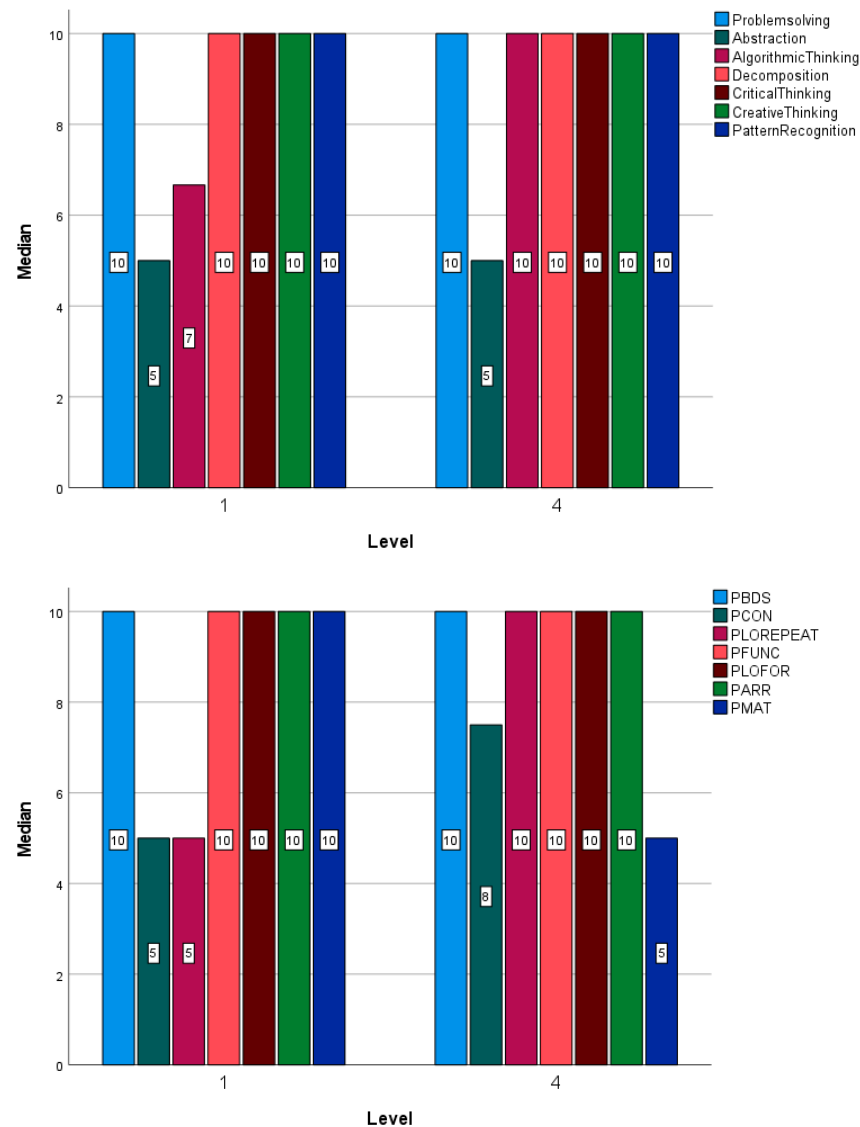


Figure 8. Bar charts of the CT skills in CTScore (**top**) and the programming test in CTProg (**bottom**) by years (1st and 4th) at URJC.

3.2.3. Atlantic Technological University (ATU)

The study will now focus on Atlantic Technological University (ATU). It is observed that, although the difference between medians is not statistically significant, the medians of the two variables, CTScore and CTProg, have lowered their value from the first to the fourth year (Figure 9). The dispersion, in the case of CTScore, has increased a lot, while remaining similar in CTProg.

In the case of the bar graphs, as shown in Figure 10, very little change is seen in the medians, depending on the course, except for the abstraction variable (Abstraction), where it decreases from five to two, the creative thinking variable (CreativeThinking), where it increases slightly, and in the pattern recognition (PatternRecognition) variable, where it decreases from 10 to 6. Regarding the programming concepts, in the case of repeat loops (PLOREPEAT) and arrays (PARR), the median has increased.

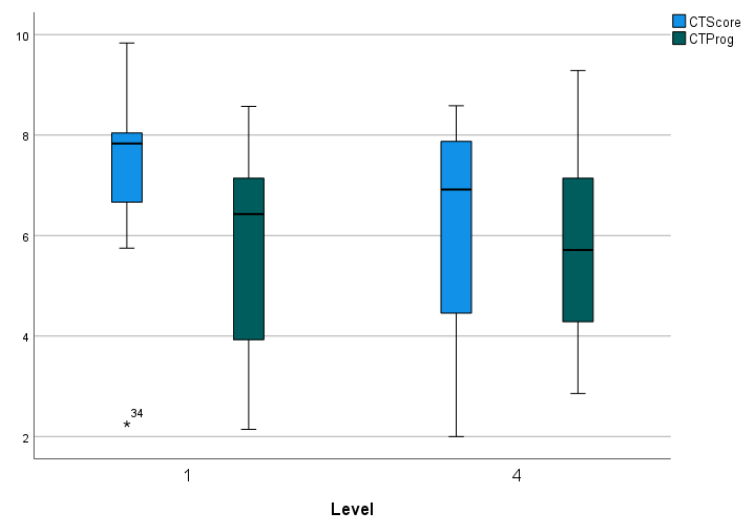


Figure 9. Box plots for the CTScore and CTProg total scores by years (1st and 4th) at ATU.

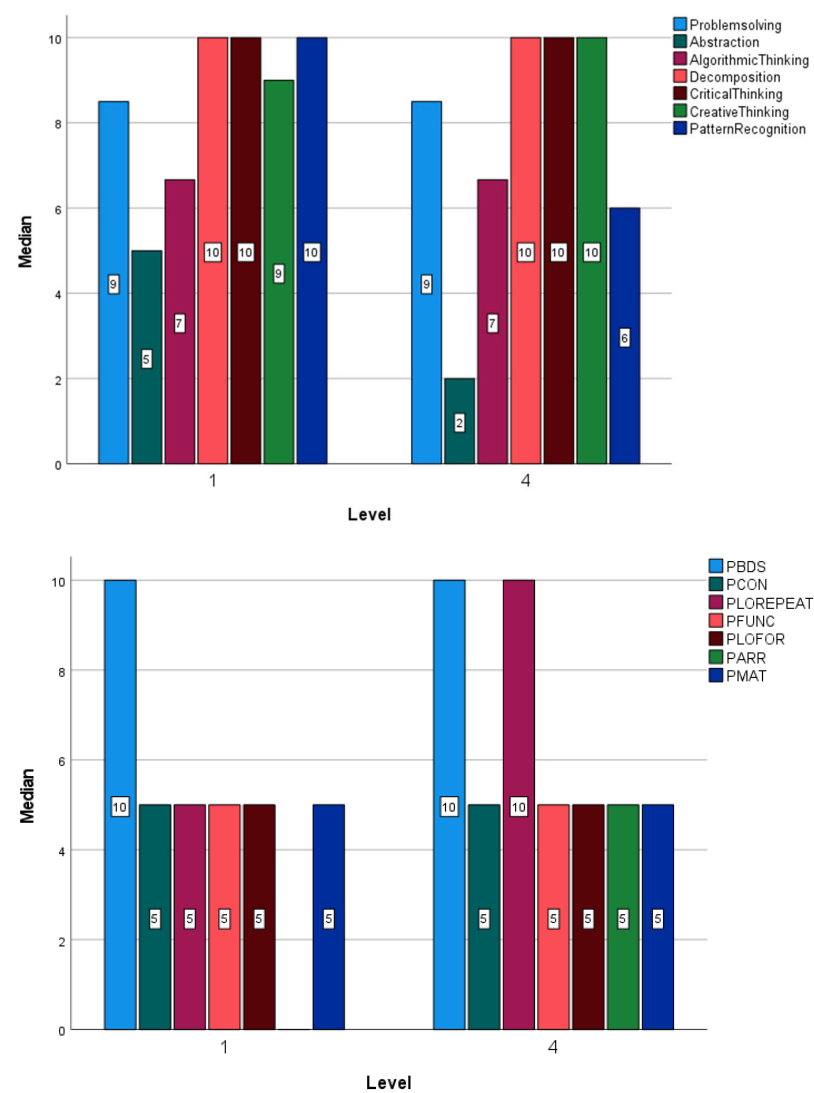


Figure 10. Bar charts of the CT skills in CTScore (top) and the programming test in CTProg (bottom) by years (1st and 4th) at ATU.

3.3. Comparison between Universities

The box plots in Figure 11 show the difference, by university, of the two global scores in CTScore and CTProg. This significant difference is manifested in significantly higher values on average at URJC (almost two points of difference in both the mean and median), with the dispersion being lower at URJC, as can be seen in Table 6 ($p < 0.001$ according to the Mann–Whitney U test, see Table 7).

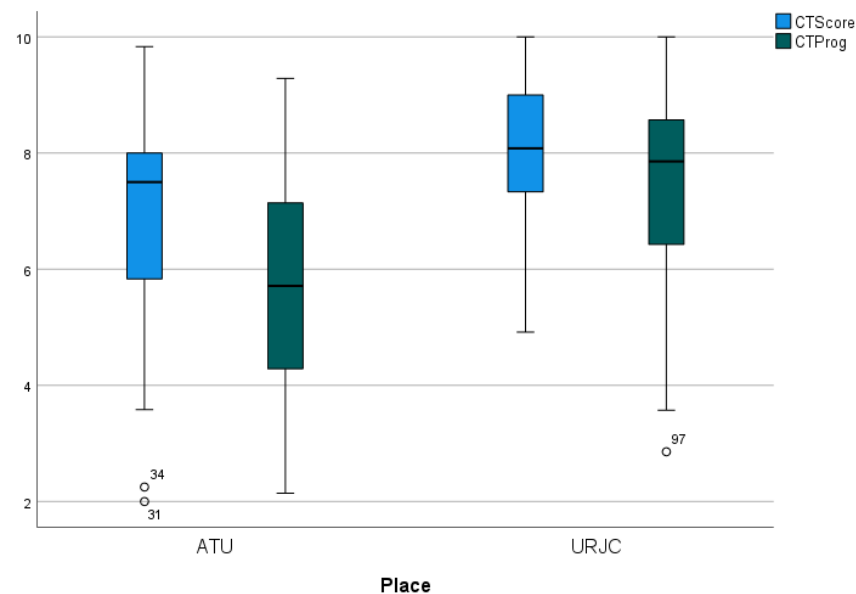


Figure 11. Box plots for the CTScore and CTProg total scores by years (1st and 4th) at both universities.

Table 6. Descriptive analysis for CTProg and CTScore by university.

	CTProg		CTScore	
	ATU	URJC	ATU	URJC
n	34	143	34	143
Mean	5.69	7.39	6.74	8.09
Median	5.71	7.85	7.5	8.08
SD	1.93	1.51	1.98	1.12

Table 7. A non-parametric test (Mann–Whitney U test) between universities.

	CTProg	CTScore
U	3642	3438
p-value	<0.001	<0.001

The median graphs show a similar trend (Figure 12), with the median values of the variables studied being greater or equal, in all cases, at URJC.

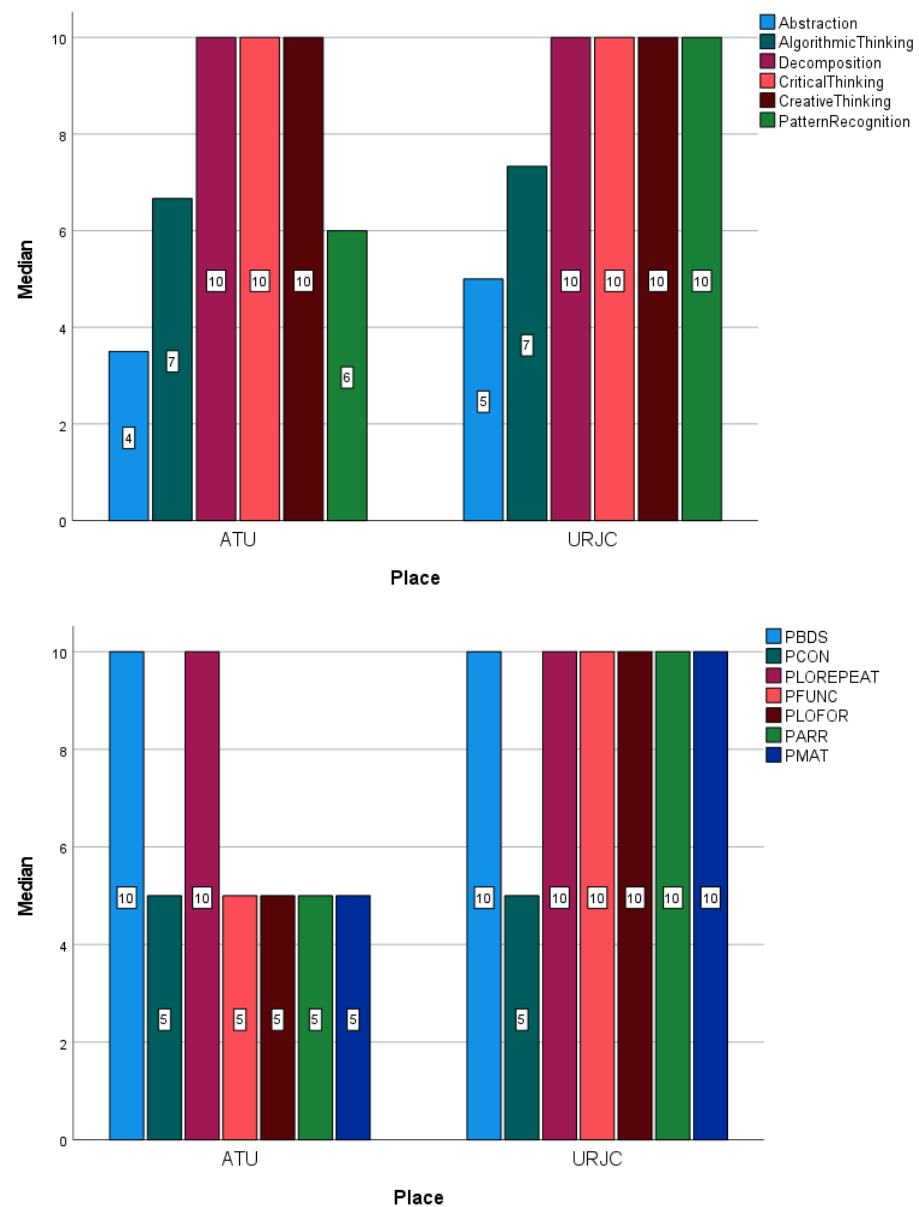


Figure 12. Bar charts of the CT skills in CTScore (**top**) and the programming test in CTProg (**bottom**) by years (1st and 4th) at ATU and URJC.

3.4. Comparison between Genders

Between genders, the box plots included in Figure 13 show the similarity in both the median and dispersion for all genders. In the case of “Other” genders, given the small quantity (two cases), their interpretation is not very relevant. Table 8 shows most relevant descriptive statistics. Higher values are shown for the male gender in terms of the mean, median, and standard deviation. Table 9 shows the output of the non-parametric Kruskal–Wallis statistic, showing that these differences are not significant ($p > 0.05$).

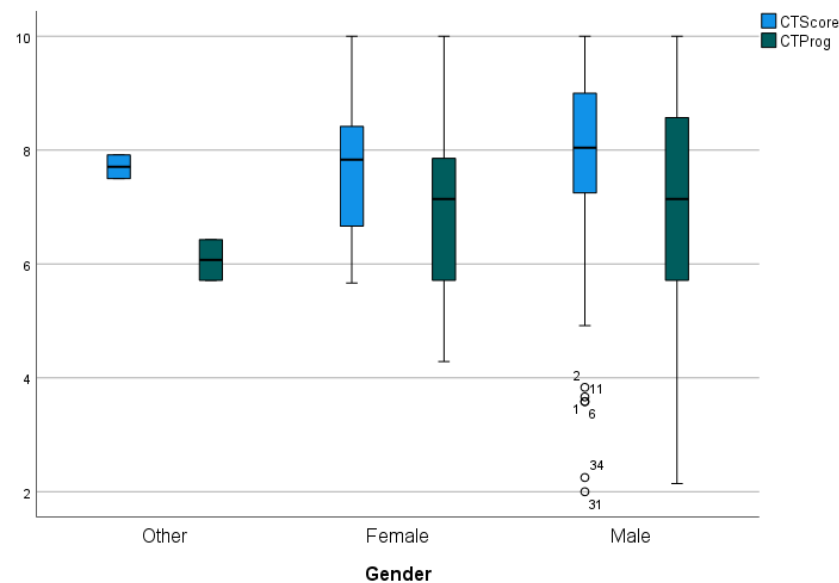


Figure 13. Box plots for the CTScore and CTProg total scores by genders at both universities.

Table 8. Descriptive analysis for CTProg and CTScore by gender.

	CTScore			CTProg		
	Other	Female	Male	Other	Female	Male
n	2	35	140	2	35	140
Mean	6.07	6.83	7.14	7.70	7.69	7.87
Median	6.07	7.14	7.14	7.70	7.83	8.04
SD	0.50	1.45	1.80	0.19	1.29	1.49

Table 9. A non-parametric test (Kruskal–Wallis test) among genders.

	CTProg	CTScore
W	3642	1.984
p-value	0.173	0.371

4. Discussion

This study offers a significant addition to the literature by providing a thorough examination of the methods for gauging the differences in students' programming and computational thinking abilities by geography, gender, and university level. This paper presents UniCTCheck, an innovative approach created to evaluate the fundamentals of computational thinking in computer science students. This study used two important tools, the web application CTScore and the psychometric scale CTProg, to measure seven fundamental aspects of computational thinking and six critical programming ideas for computer science students. As a result, the following five research questions were formulated:

4.1. RQ1: Does UniCTCheck Measure the Main Components of Computational Thinking and Programming for CS University Students?

The results from the study indicate that UniCTCheck effectively measures the main components of computational thinking and programming for computer science university students. Through the use of CTScore and CTProg, a comprehensive assessment of pattern recognition, creative thinking, algorithmic thinking, problem solving, critical thinking, decomposition, and abstraction, as well as programming concepts like basic directions and sequences, conditionals, loops, functions, and data structures was conducted. The data reveal that the UniCTCheck method provides a robust framework for evaluating students' computational thinking abilities and programming skills accurately.

4.2. RQ2: Is There a Relationship between Students' Computational Thinking Skills and Programming Abilities?

The time expended by students in answering the programming test (CTProg) was directly related to their higher performance on it. Also, students that did well on the CTProg also did well on the CT skills test (CTScore); thus, it can be said that programming knowledge leads to CT skills acquisition. It has also been proved that students learning any type of loop leads to their learning of conditionals, and vice versa. Furthermore, students that know of more advanced programming concepts such as arrays need to have previously mastered conditionals and loops. Moreover, the comprehension of two-dimensional arrays (matrices) required the knowledge and understanding of one-dimensional arrays and for loop structures so to work effectively with them. Regarding CT skills acquisition, students with the problem-solving ability did well on the algorithmic thinking ability, and vice versa. In the same manner, students with a higher capacity of abstraction performed as well or better in other CT skills such as algorithmic thinking, decomposition, and creative thinking. As shown, students overperforming in algorithmic thinking also did well in CT skills like decomposition and pattern recognition.

4.3. RQ3: Do Students' Computational Thinking Skills and Programming Abilities Vary by University Level?

In general, there were no significant differences between both courses. Although they were not statistically significant, changes can be seen between the first and fourth years for the programming concepts of repeat loops, functions, arrays, and matrices.

When the study focused on Rey Juan Carlos University (URJC), in the case of CTScore, the median increased slightly in the fourth year, while the dispersion also clearly increased. In the case of CTProg, the median also increased from the first to the fourth year, and the dispersion in the data increased again. Also, very little change occurred in the medians, depending on the course, except in the algorithmic thinking skill from CTScore. In the case of the programming variables, the median increased in the conditionals and repeat loops and decreased in the matrices.

When the study focused on Atlantic Technological University (ATU), it was observed that, although the difference between medians was not statistically significant, the medians of the two variables, CTScore and CTProg, lowered their values from the first to the fourth year. The dispersion, in the case of CTScore, increased a lot, while remaining similar in CTProg. Very little change was seen in the medians, depending on the course, except in the case of the CT skills of abstraction, where it decreased, in creative thinking, where it increased slightly, and in pattern recognition, where it decreased. Regarding, the programming concepts, in the case of repeat loops and arrays, the median increased.

4.4. RQ4: Do Students' Computational Thinking Skills and Programming Abilities Vary by University Location?

There existed a significant difference by university in the two global scores in CTScore and CTProg. This significant difference was manifested in significantly higher values on average (almost two points of difference in both the mean and median), with the dispersion being lower at URJC. Also, the median values of the variables studied were greater or equal, in all cases, at URJC.

4.5. RQ5: Do Students' Computational Thinking Skills and Programming Abilities Vary by Gender?

The examination of computational thinking skills and programming abilities by gender revealed no statistically significant difference. While there were minor variations in the performance between male and female students, the data indicated that both genders displayed similar levels of competency overall. This suggests that computational thinking skills and programming abilities are not inherently gender-specific and can be developed by students regardless of gender.

In conclusion, the data-driven responses to the research questions provide valuable insights into the nuances of students' computational thinking skills and programming abilities in the context of gender, university level, and geography. By leveraging the UniCTCheck method and the assessment tools, CTScore and CTProg, this study has shed light on the diverse competencies of computer science students and highlighted the importance of tailored educational interventions to enhance their skills effectively.

5. Limitations and Future Research

The greatest limitation of the study is related to the impossibility of the same professors being present in all study groups at the different universities. This introduces the human factor in terms of the personal motivation with which students have been asked to participate. To avoid this possible bias, we always tried to follow the same guidelines in terms of time, explanation, motivation, etc. Furthermore, different languages have been considered and the tests were translated into both English and Spanish. Another limitation between the results from both university degrees may be the difference in the admission requirements between the two programs, with the requirements being much higher at URJC.

Future research may include more difficult questions on the UniCTCheck, so the ceiling can be a little bit higher, and therefore more differences can be found. Also, future evaluation with non-CS students should be considered.

6. Conclusions

This study significantly advances the understanding of how to measure differences in computational thinking (CT) and programming abilities among computer science (CS) students by geography, gender, and university level. The innovative UniCTCheck method, incorporating the CTScore web application and CTProg psychometric scale, effectively evaluates seven fundamental CT components and six critical programming concepts.

UniCTCheck Effectiveness: UniCTCheck was validated as a robust framework for assessing CT components and programming skills in CS students. The comprehensive evaluation demonstrated its effectiveness in measuring essential skills such as pattern recognition, algorithmic thinking, and problem solving, as well as programming concepts including loops, functions, and data structures.

Relationship Between CT Skills and Programming Abilities: A significant correlation was found between students' CT skills and programming abilities. Higher performance in the CTProg programming test was directly related to better scores in the CTScore assessment, indicating that strong programming knowledge facilitates CT skills acquisition.

Variation by University Level: The study observed minimal differences in CT skills and programming abilities across university levels. However, slight increases in median scores for advanced programming concepts were noted in fourth-year students, particularly at Rey Juan Carlos University (URJC), where there was an increased dispersion in both the CTScore and CTProg results.

Variation by University Location: Significant differences were found between universities, with URJC students outperforming those at Atlantic Technological University (ATU) on both CTScore and CTProg. URJC students displayed higher mean and median scores with a lower dispersion. As explained in the limitation section, this may be due to the difference in admission requirements in between the two programmes.

Variation by Gender: No statistically significant differences were found in CT skills and programming abilities between genders. Both male and female students demonstrated similar levels of competency, indicating that these skills are not inherently gender-specific.

In summary, this study provides valuable insights into the factors influencing CS students' CT and programming abilities. The use of UniCTCheck, CTScore, and CTProg offers a comprehensive assessment framework that can inform tailored educational interventions to enhance students' computational competencies effectively.

Author Contributions: Conceptualisation, R.H.-N., C.P., J.F., D.P.-A. and E.Ç.; methodology, R.H.-N., C.P., J.F., D.P.-A. and E.Ç.; software, R.H.-N., D.P.-A. and E.Ç.; validation, R.H.-N., C.P., J.F. and E.Ç.; formal analysis, R.H.-N. and C.P.; investigation, R.H.-N., C.P. and J.F.; resources, R.H.-N., D.P.-A. and E.Ç.; data curation, R.H.-N. and C.P.; writing—original draft preparation, R.H.-N. and C.P.; writing—review and editing, R.H.-N., C.P. and J.F.; visualisation, R.H.-N., C.P., D.P.-A. and E.Ç.; supervision, R.H.-N. All authors have read and agreed to the published version of the manuscript.

Funding: This research was funded by research grants PID2022-137849OB-I00 funded by MICIU/AEI/10.13039/501100011033 and by the ERDF, EU.

Institutional Review Board Statement: The study was conducted in accordance with the Declaration of Helsinki, and approved by Research Ethics Committee of Rey Juan Carlos University with internal registration number 1909202332223, 9 October 2023.

Informed Consent Statement: Informed consent was obtained from all subjects involved in the study.

Data Availability Statement: The data presented in this study are available on request from the corresponding author due to ethical reasons.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Wing, J.M. Computational thinking. *Commun. ACM* **2006**, *49*, 33–35. [CrossRef]
2. Wing, J.M. Computational thinking and thinking about computing. *Philos. Trans. R. Soc. A Math. Phys. Eng.* **2008**, *366*, 3717–3725. [CrossRef] [PubMed]
3. Cuny, J.; Snyder, L.; Wing, J.M. Demystifying computational thinking for non-computer scientists. *Educ. Res. Rev.* **2010**, *22*, 142–158.
4. Mohaghegh, M.; McCauley, M. Computational Thinking: The Skill Set of the 21st Century. *Int. J. Comput. Sci. Inf.* **2016**, *7*, 1524–1530.
5. Fisher, J.; Henzinger, T.A. Executable cell biology. *Nat. Biotechnol.* **2007**, *25*, 1239–1249. [CrossRef] [PubMed]
6. Qin, H. Teaching computational thinking through bioinformatics to biology students. In Proceedings of the 40th ACM Technical Symposium on Computer Science Education, New York, NY, USA, 4–7 March 2009.
7. García-Peñalvo, F.J.; Mendes, A.J. Exploring the computational thinking effects in pre-university education. *Comput. Hum. Behav.* **2018**, *80*, 407–411. [CrossRef]
8. Higgins, C.; O’Leary, C.H.; OMtenzi, F. A conceptual framework for a software development process based on computational thinking. In Proceedings of the 11th International Technology, Education and Development Conference (INTED17), Valencia, Spain, 6–8 March 2017; pp. 455–464. [CrossRef]
9. Selby, C.; Woollard, J. *Computational Thinking: The Developing Definition*; University of Southampton Institutional Research Repository: Swindon, UK, 2013; 6p. Available online: <http://eprints.soton.ac.uk/id/eprint/356481> (accessed on 10 June 2024).
10. de Araujo, A.L.S.O.; Andrade, W.L.; Guerrero, D.D.S. A systematic mapping study on assessing computational thinking abilities. In Proceedings of the 2016 IEEE Frontiers in Education Conference (FIE), Erie, PA, USA, 12–15 October 2016; IEEE: Piscataway, NJ, USA, 2016; pp. 1–9.
11. Kong, S.-C. Components and methods of evaluating computational thinking for fostering creative problem-solvers in senior primary school education. In *Computational Thinking Education*; Springer: Berlin/Heidelberg, Germany, 2019; pp. 119–141. [CrossRef]
12. Román-González, M.; Moreno-León, J.; Robles, G. Combining assessment tools for a comprehensive evaluation of computational thinking interventions. In *Computational Thinking Education*; Springer: Berlin/Heidelberg, Germany, 2019; pp. 79–98.
13. Denning, P.J. Remaining trouble spots with computational thinking. *Commun. ACM* **2017**, *60*, 33–39. [CrossRef]
14. Haseski, H.I.; Ilic, U. An Investigation of the Data Collection Instruments Developed to Measure Computational Thinking. *Inform. Educ.* **2019**, *18*, 297. [CrossRef]
15. Tang, X.; Yin, Y.; Lin, Q.; Hadad, R.; Zhai, X. Assessing computational thinking: A systematic review of empirical studies. *Comput. Educ.* **2020**, *148*, 103798. [CrossRef]
16. Varghese, V.V.; Renumol, V.G. Assessment methods and interventions to develop computational thinking—A literature review. In Proceedings of the 2021 International Conference on Innovative Trends in Information Technology (ICITIIT), Kottayam, India, 11–12 February 2021; IEEE: Piscataway, NJ, USA, 2021; pp. 1–7.
17. Poulakis, E.; Politis, P. Computational Thinking Assessment: Literature Review. In *Research on E-Learning and ICT in Education: Technological; Pedagogical and Instructional Perspectives*; Springer: Berlin/Heidelberg, Germany, 2021; pp. 111–128.
18. Alan, Ü. Likert Tipi Ölçeklerin Çocuklarla Kullanımında Yanıt Kategori Sayısının Psikometrik Özelliklere Etkisi. Master’s Thesis, Hacettepe University, Ankara, Turkey. (In Turkish).
19. Arıkan, R. Anket yöntemi üzerinde bir değerlendirme. *Haliç Üniversitesi Sos. Bilim. Derg.* **2018**, *1*, 97–159. (In Turkish)
20. Guenaga, M.; Eguíluz, A.; Garaizar, P.; Gibaja, J. How do students develop computational thinking? Assessing early programmers in a maze-based online game. *Comput. Sci. Educ.* **2021**, *31*, 259–289. [CrossRef]

21. Kong, S.-C.; Liu, B. A performance-based assessment platform for developing computational thinking concepts and practices: EasyCode. *Bull. Tech. Comm. Learn. Technol.* **2020**, *20*, 3–10. ISSN 2306-0212. Available online: <https://ieeecs-media.computer.org/tc-media/sites/5/2020/06/07190536/bulletin-tclt-2020-0201001.pdf> (accessed on 10 June 2024).
22. Bikmaz-Bilgen, Ö. Tamamlayıcı Ölçme ve Değerlendirme Teknikleri II: Portfolyo Değerlendirme. In *Eğitimde Ölçme ve Değerlendirme*; Dogan, N., Ed.; Pegem Akademi: Ankara, Turkey, 2019; pp. 182–214. (In Turkish)
23. Gültekin, S. Performans Dayanlı Değerlendirme. In *Eğitimde Ölçme ve Değerlendirme*; Demirtasli, R.N., Ed.; Ani Pub: Ankara, Turkey, 2017; pp. 233–256. (In Turkish)
24. Sahin, M.G. Performansa Dayalı Değerlendirme. In *Eğitimde Ölçme ve Değerlendirme*; Çetin, B., Ed.; Ani Pub: Ankara, Turkey, 2019. (In Turkish)
25. Grover, S.; Pea, R. Computational thinking in K–12: A review of the state of the field. *Educ. Res.* **2013**, *42*, 38–43. [CrossRef]
26. Mannila, L.; Dagiene, V.; Demo, B.; Grgurina, N.; Mirolo, C.; Rolandsson, L.; Settle, A. Computational thinking in K-9 education. In Proceedings of the Working Group Reports of the 2014 on Innovation & Technology in Computer Science Education Conference, Uppsala, Sweden, 23–25 June 2014. [CrossRef]
27. Martins-Pacheco, L.H.; von Wangenheim, C.A.G.; Alves, N. Assessment of computational thinking in K-12 context: Educational practices, limits and possibilities—a systematic mapping study. In Proceedings of the 11th International Conference on Computer Supported Education (CSEDU 2019), Crete, Greece, 2–4 May 2019; pp. 292–303.
28. Çoban, E.; Korkmaz, Ö. An alternative approach for measuring computational thinking: Performance-based platform. *Think. Ski. Creat.* **2021**, *42*, 100929. [CrossRef]
29. Tedre, M.; Denning, P.J. The long quest for computational thinking. In Proceedings of the 16th Koli Calling International Conference on Computing Education Research, Koli, Finland, 24–27 November 2016; pp. 120–129. [CrossRef]
30. Katz, D.L. Conference report on the use of computers in engineering classroom instruction. *Commun. ACM* **1960**, *3*, 522–527. [CrossRef]
31. Newell, A.; Perlis, A.J.; Simon, H.A. Computer science. *Science* **1967**, *157*, 1373–1374. [CrossRef] [PubMed]
32. Papert, S. *Mindstorms: Children, Computers, and Powerful Ideas*; Basic Books, Inc.: New York, NY, USA, 1980.
33. Papert, S. An exploration in the space of mathematics educations. *Int. J. Comput. Math. Learn.* **1996**, *1*, 95–123. [CrossRef]
34. Papert, S.; Harel, I. Situating constructionism. *Constructionism* **1991**, *36*, 1–11.
35. Turkle, S.; Papert, S. Epistemological pluralism: Styles and voices within the computer culture. *Signs J. Women Cult. Soc.* **1990**, *16*, 128–157. [CrossRef]
36. Denning, P.J. The profession of IT beyond computational thinking. *Commun. ACM* **2009**, *52*, 28–30. [CrossRef]
37. Denning, P.J. Ubiquity symposium ‘what is computation?’: Opening statement. *Ubiquity* **2010**, *2010*, 1. [CrossRef]
38. Easton, T.A. Beyond the algorithmization of the sciences. *Commun. ACM* **2006**, *49*, 31–33. [CrossRef]
39. Dijkstra, E.W. Programming as a discipline of mathematical nature. *Am. Math. Mon.* **1974**, *81*, 608–612. [CrossRef]
40. Guzdial, M. Learner-centered design of computing education: Research on computing for everyone. In *Synthesis Lectures on Human-Centered Informatics (SLHCI)*; Springer: Berlin/Heidelberg, Germany, 2015; Volume 8, pp. 1–165. [CrossRef]
41. Lee, I.; Martin, F.; Denner, J.; Coulter, B.; Allan, W.; Erickson, J.; Malyn-Smith, J.; Werner, L. Computational thinking for youth in practice. *ACM Inroads* **2011**, *2*, 32–37. [CrossRef]
42. Shute, V.J.; Sun, C.; Asbell-Clarke, J. Demystifying computational thinking. *Educ. Res. Rev.* **2017**, *22*, 142–158. [CrossRef]
43. Lu, J.J.; Fletcher, G.H.L. Thinking about computational thinking. In Proceedings of the 40th ACM Technical Symposium on Computer Science Education—SIGCSE’09, Chattanooga, TN, USA, 4–7 March 2009. [CrossRef]
44. Aho, A.V. Computation and computational thinking. *Comput. J.* **2012**, *55*, 832–835. [CrossRef]
45. Kong, S.-C.; Abelson, H.; Lai, M. Introduction to computational thinking education. In *Computational Thinking Education*; Springer: Berlin/Heidelberg, Germany, 2019; pp. 1–10. [CrossRef]
46. Andrew, C.; Paul, C.; Mark, D.; Simon, H.; Thomas, N.; Cynthia, S.; John, W. *Computational Thinking—A Guide for Teachers; Computing at School*; University of Southampton Institutional Research Repository: Swindon, UK, 2015. Available online: <https://eprints.soton.ac.uk/424545/> (accessed on 11 May 2020).
47. CSTA. *Operational Definition of Computational Thinking for K–12 Education*; National Science Foundation: Alexandria, VA, USA, 2011.
48. ISTE Standards for Students. Available online: <http://www.iste.org/standards/standards/for-students-2016> (accessed on 14 April 2020).
49. Riley, D.D.; Hunt, K.A. *Computational Thinking for the Modern Problem Solver*; CRC Press: Boca Raton, FL, USA, 2014; ISBN 978-1-4665-8777-9.
50. NRC. Committee for the workshops on computational thinking. In *Report of a Workshop on the Scope and Nature of Computational Thinking*; National Academies Sciences, Engineering, Medicine: Washington, DC, USA, 2010.
51. Barr, V.; Stephenson, C. Bringing computational thinking to K-12: What is involved and what is the role of the computer science education community? *ACM Inroads* **2011**, *2*, 48–54. [CrossRef]
52. Brennan, K.; Resnick, M. New frameworks for studying and assessing the development of computational thinking. In Proceedings of the 2012 Annual Meeting of the American Educational Research Association, Vancouver, BC, Canada, 13–17 April 2012; p. 25. Available online: <http://scratched.gse.harvard.edu/ct/files/AERA2012.pdf> (accessed on 10 June 2024).

53. Gouws, L.A.; Bradshaw, K.; Wentworth, P. Computational thinking in educational activities: An evaluation of the educational game light-bot. In Proceedings of the 18th ACM Conference on Innovation and Technology in Computer Science Education, Canterbury, UK, 1–3 July 2013; pp. 10–15. [[CrossRef](#)]
54. Bers, M.U.; Flannery, L.; Kazakoff, E.R.; Sullivan, A. Computational thinking and tinkering: Exploration of an early childhood robotics curriculum. *Comput. Educ.* **2014**, *72*, 145–157. [[CrossRef](#)]
55. Zapata-Ros, M. Pensamiento Computacional: Una Nueva Alfabetización Digital. *Distance Educ. J.* **2015**, *46*. Available online: <https://revistas.um.es/red/article/view/240321> (accessed on 10 June 2024). [[CrossRef](#)]
56. Anderson, N.D. A call for computational thinking in undergraduate psychology. *Psychol. Learn. Teach.* **2016**, *15*, 226–234. [[CrossRef](#)]
57. Google for Education: Computational Thinking. Available online: <https://edu.google.com/resources/programs/exploring-computational-thinking/> (accessed on 21 February 2020).
58. Dierbach, C.; Hochheiser, H.; Collins, S.; Jerome, G.; Ariza, C.; Kelleher, T.; Kleinsasser, W.; Dehlinger, J.; Kaza, S. A model for piloting pathways for computational thinking in a general education curriculum. In Proceedings of the 42nd ACM Technical Symposium on Computer Science Education, Dallas, TX, USA, 9–12 March 2011; pp. 257–262. [[CrossRef](#)]
59. Berland, M.; Lee, V.R. Collaborative strategic board games as a site for distributed computational thinking. *Int. J. Game-Based Learn.* **2011**, *1*, 65–81. [[CrossRef](#)]
60. Akkaya, A. The Effects of Serious Games on Students’ Conceptual Knowledge of Object-Oriented Programming and Computational Thinking Skills. Master’s Thesis, Bogaziçi University, Beşiktaş, Turkey, 2018.
61. CAS. Available online: <https://community.computingatschool.org.uk/resources/2324> (accessed on 10 June 2024).

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.